Strathmore
UNIVERSITY

School of Computing and Engineering Sciences

BBT 1201: Object-Oriented Programming

Mini-project guide

191368 - Nzuki Linus Wambua

225366 - Mwema Ian Mwendwa

Instructions

1. This project is to be ***done in pairs***, but ***each member is required to submit their work***.
2. Answer every question on this document in the spaces provided.
3. Maintain the document format when answering especially when writing code.
4. Provide additional explanations where you feel diagrams may not be sufficient.
5. Once you are done answering, ***convert the document to .pdf format*** for submission.

Objectives:

To design and implement a simple Management System using:

- Java (Object-Oriented Programming principles)
- PostgreSQL Database
- JDBC
- Swing (GUI)

Project Overview

The main idea of this OOP Mini Project is to assess your understanding and practical application of key Object-Oriented Programming concepts through the development of a complete, functional system. In this project, you are required to design and implement a Simple Management System of your choice using Java, PostgreSQL, JDBC, and Swing, demonstrating your understanding of object-oriented programming concepts, JDBC database connectivity, GUI development, and UML system design. The system should simulate a real-world management scenario such as a Student, Library, Inventory, Employee, or Hospital Management System (can be based on the object that you identified in the previous assignments), and it must support full CRUD operations: insert, update, delete, select (single record), and selectAll (multiple records), implemented using the abstract class structure specified in the project requirements.

Design Requirements

You must design and submit a UML diagram showing:

- All classes
- Attributes
- Methods
- Relationships
- Inheritance structure

The UML must clearly show:

- DBConnection (abstract class)
- DBOperations (child class)
- GUI classes
- Model classes

Submit the UML diagram as a PDF or image file.

Technical Requirements

- You must use **PostgreSQL** as your database.
- You must create **at least** one main table relevant to your system.
- Your system must connect to the database using JDBC following the two-step connection process taught in class (can be a continuation of the database assignment).
- You must create an abstract class named `DBConnection` that:
    1. Contains a global variable of type Connection
    2. Initialize the connection to your database inside the constructor
    3. Load the PostgreSQL driver inside the constructor
    4. Establish the database connection using `DriverManager.getConnection()`
    5. Declares five abstract methods: `insert()`, `update()`, `delete()`, `select()` and `selectAll()` (add relevant parameters as required).
- You must create a class named DBOperations that extends DBConnection and implements ALL five abstract methods.
- You must create a class (JFrame) for each GUI.
- You are free to create additional classes such as: Model classes (e.g., Student, Product, Employee), Utility classes and a Main class)
- Your system must include a login screen that directs the user to the main screen (among others), upon successful login.
- **Your system must clearly demonstrate proper use of OOP principles**.

Presentation requirements:

1. Complete the system.
2. Prepare a 3 – 5-minute video presenting your system. The face of the two presenters must be seen at the beginning of the video, and each should present their contribution.

3. Explanation of: System design, UML diagram, database structure (you can use a database schema), OOP concepts applied, implementation of CRUD methods and a live demonstration of the working system.
4. **Upload the video your google drive**, using the school provided email address. **Share a link to the lecturer's email** (*jmido@strathmore.edu*) with the **subject** 000000 and 000000 – OOP Mini Project Presentation.
5. Submit this document to your elearning.
6. **Bonus:** Explain unique features that you have added, explain proper exception handling, input validation, clean UI Design and showing the architecture.
7. **Optional:** You can slightly modify the initial requirements to better fit an original idea that you have thought of implementing, for the Bonus marks section (*it has a higher percentage*).

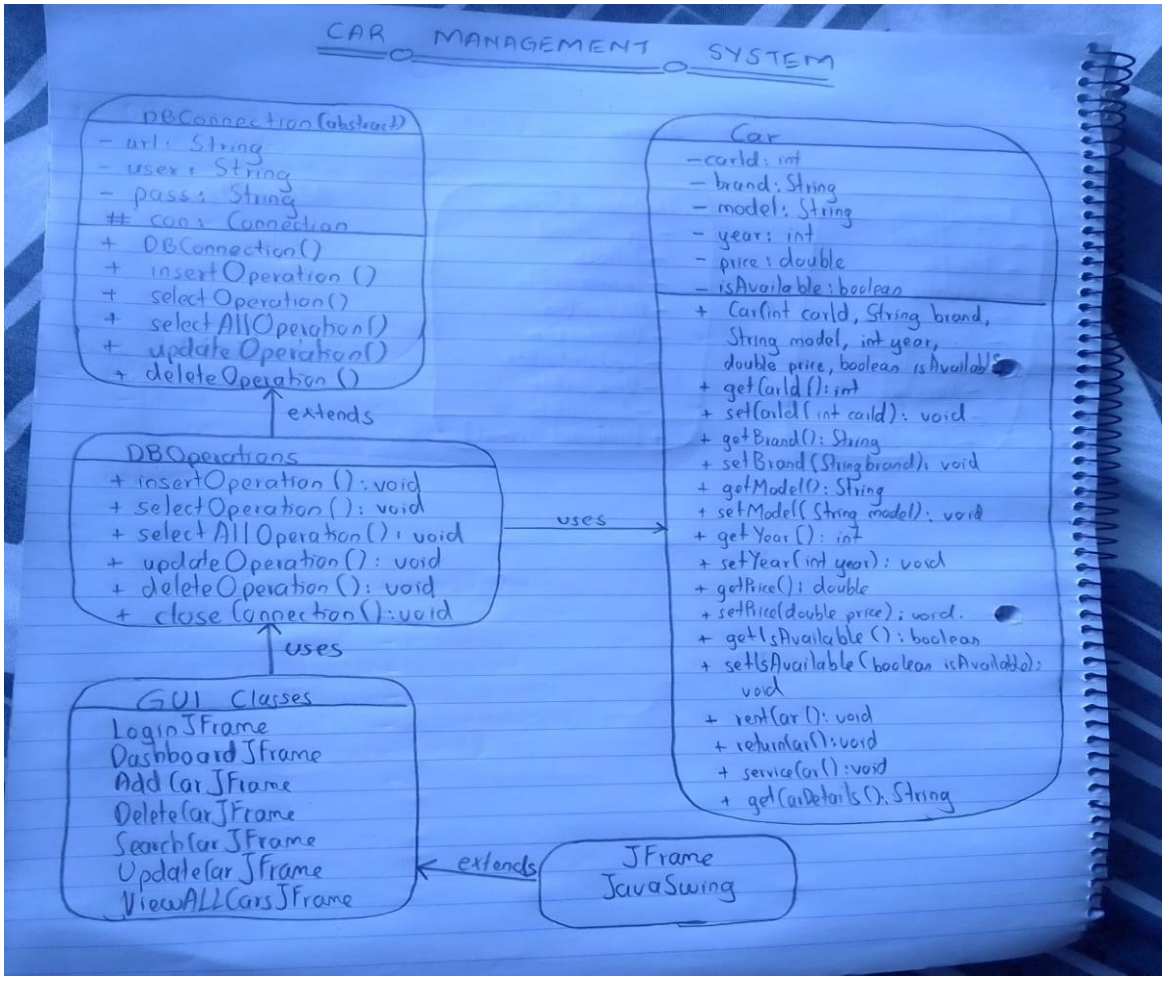
Once you are done, submit this document on eLearning. The deadline for sending the presentation video will be on the same day as submitting this document.

Submission instructions:

1. Answer the questions in this document and submit it on eLearning, after converting it to ***.pdf*** *format.*
2. Upload **zipped project folder** on eLearning, with appropriate title.
3. Upload the video to your Google Drive (***only one member is required to upload***). Video should not be less than 3 minutes or more than 5 minutes.

Questions

1. Upload your UML class diagram.


CAR MANAGEMENT SYSTEM
DBConnection (abstract)
- url: String
- user: String
- pass: String
con: Connection
+ DBConnection()
+ insertOperation()
+ selectOperation()
+ selectAllOperation()
+ updateOperation()
+ deleteOperation()
extends
DBOperations
+ insertOperation(): void
+ selectOperation(): void
+ selectAllOperation(): void
+ updateOperation(): void
+ deleteOperation(): void
+ closeConnection(): void
uses
uses
Car
- carId: int
- brand: String
- model: String
- year: int
- price: double
- isAvailable: boolean
+ Car(int carId, String brand, String model, int year, double price, boolean isAvailable)
+ getCarId(): int
+ setCarId(int carId): void
+ getBrand(): String
+ setBrand(String brand): void
+ getModel(): String
+ setModel(String model): void
+ getYear(): int
+ setYear(int year): void
+ getPrice(): double
+ setPrice(double price): void
+ getIsAvailable(): boolean
+ setIsAvailable(boolean isAvailable): void
+ rentCar(): void
+ returnCar(): void
+ serviceCar(): void
+ getCarDetails(): String
GUI Classes
LoginJFrame
DashboardJFrame
AddCarJFrame
DeleteCarJFrame
SearchCarJFrame
UpdateCarJFrame
ViewALLCarsJFrame
extends
JFrame
JavaSwing